10-minute developer presentation

Your focus: how the application was built, why the stack was selected, how frontend and backend communicate, and what users can do. Database coverage is limited to tables, keys and integrity. No ER-diagram discussion.

Time	Topic
0:00–0:45	Project introduction and your contribution
0:45–2:15	Development stack and why each tool is used
2:15–4:00	Frontend architecture, components, hooks and routing
4:00–5:30	Backend integration, update flow and authentication
5:30–7:30	Admin tabs and student portal walkthrough
7:30–8:30	SQL viewer and one live development example
8:30–9:00	Brief database tables, PKs and FKs
9:00–10:00	Codebase management, verification and closing

1. Introduction · 45 seconds

> “CampusOS is a college management application with separate administrator and student experiences. My contribution focuses on the frontend, backend integration and codebase organization. I built the interface using reusable React components, connected forms and tables to server APIs, and handled loading, errors and sessions. I will explain the development decisions and then demonstrate how an administrator’s update becomes visible in the student portal.”

Present the Next.js/SQLite build running on port 3000. The repository also contains a separate React/Vite + FastAPI + PostgreSQL version. These are independent applications with similar academic API interfaces, not two frontends connected to the same live database. The extended portal described here belongs to the Next build.

0:45–2:15 / STACK CHOICES

2. Technology and why we used it

Technology	What it does	Why it fits this project
JavaScript + JSX	Application logic and React markup.	JSX expresses UI as components; JavaScript is used across browser and Node backend. This build is not TypeScript.
React 19	Renders screens from state and props.	Reusable components avoid separate implementations for every management table and form.
Next.js 15 App Router	Application routes, React hosting and server API routes.	Frontend and backend can run in one project and origin, simplifying the local demo and cookie-based integration.
Tailwind CSS 3	Utility classes and shared theme tokens.	Consistent spacing, typography, colors and responsive layouts without duplicating CSS for every page.
Axios	Shared HTTP client for JSON APIs.	One configured instance centralizes API base URL, interceptors, session expiry and safe read retries.
React Hook Form	Form state and field validation.	Reusable forms handle required inputs, errors and submission without writing separate state handlers for every field.
Recharts	Dashboard bar, line and pie charts.	React-friendly chart components turn API results into clear summaries.
Lucide React	Icons.	Consistent visual language across sidebar, actions and status controls.
Node.js + node:sqlite	Server-side execution and SQL access.	Direct prepared statements, no ORM needed in this build. A local SQLite file avoids a separate database server.
Git / GitHub	Version history and remote repository.	Traceable changes, reproducible source and sharing. Generated local databases are excluded.
Node test runner + Puppeteer	API checks and real Chromium UI checks.	Tests verify behavior below and above the HTTP boundary.

> “React is the UI library; Next.js is the framework around it. Node runs the server. Axios transports requests, while SQLite stores data. Each tool has a separate responsibility. The design uses charcoal surfaces and a flat green accent, with shared tokens so appearance stays consistent.”

Trade-off: node:sqlite calls are synchronous and SQLite has one writer at a time. It is practical for a portable local project, not a claim of unlimited production scalability.

2:15–4:00 / FRONTEND ARCHITECTURE

3. How the frontend is made

> “The application is component-based. CollegeApp checks the session and chooses the correct experience. AppLayout provides the admin shell. Most management screens share one ResourcePage component, configured with different columns, fields and endpoints. This reduces repetition and makes fixes consistent across entities.”

File / concept	Responsibility and explanation
CollegeApp.jsx	Loads /auth/me; shows a connecting state, Login, the student portal or the admin application. Maps the current URL to a screen.
AppLayout, Sidebar, Topbar	Shared navigation, responsive shell, sidebar collapse, mobile navigation and theme controls.
config/resources.js	Declares resource endpoints, columns, form fields, filters, default sort and delete warnings. Configuration is separate from rendering logic.
ResourcePage.jsx	Manages list loading, search, filters, page state, create/edit forms, confirmation and refresh after CRUD.
DataTable / Pagination / Badge	Reusable row presentation, page controls and visual status labels.
FormModal / Field / ConfirmDialog	Shared forms and safe delete confirmation rather than duplicating markup and behavior.
Skeleton / EmptyState / ErrorState	Different UI for waiting, no records and failed requests. Users should never mistake an error for an empty database.
services/api.js	Central Axios client, common error handling and retry/session-expiry behavior.

Hooks and state — explain confidently

- **useState:** local UI state such as open modal, current tab, page number, loading and error values.
- **useEffect:** fetch data when a screen mounts or a dependency changes; cleanup prevents old responses from updating an inactive screen.
- **useMemo:** derive a stable parameter object from search/filter/page state. **useCallback:** reuse callback references where appropriate.
- **Props:** data/configuration flows from parent components into reusable children. **Context:** ToastProvider shares notifications without threading callbacks through every component.
- **Debouncing:** waits briefly after search typing instead of sending a request for every keystroke.

Routing: Next App Router’s catch-all page hosts the application; the current pathname selects list/detail/report screens. Interactive components run on the client; database code remains on the server.

4:00–5:30 / BACKEND INTEGRATION

4. What happens when I click Save?

```
Form → Axios request → Next API route
     → session + role check → validation → prepared SQL
     → persisted row → JSON response → toast + list refetch
```

> “Typing changes form state, not the database. On Save, the form submits JSON to the API. The server validates both identity and input, writes the record and returns a response. On success the UI closes the form, invalidates cached dropdown options and refetches the list. Other pages see that update on their next data load.”

HTTP operation	Meaning
GET /api/students	Read records; request parameters describe search, filters, sort and pagination.
POST /api/students	Create a record from a JSON body.
PUT /api/students/id	Update the selected record.
DELETE /api/students/id	Delete after user confirmation, if relationships permit it.

Server files

`app/api/[...path]/route.js` receives HTTP requests. `lib/portal.mjs` checks sessions/roles and handles portal, attendance, library and account endpoints. `lib/api.mjs` handles academic CRUD, validation and snapshot generation. `queries.mjs` holds joined academic reads; `reports.mjs` builds report results.

Security is server-side, not just hidden buttons

Passwords use salted scrypt hashes. Random session tokens travel in HttpOnly, SameSite=Strict cookies; the database stores token hashes. Sessions expire after eight hours. The server rejects unauthorized roles, checks cross-origin mutations and throttles login attempts. A student cannot gain admin access by manually typing an admin URL.

Errors and reliability

401 means authentication needed; 403 means role denied; 409 means a conflict such as duplicate data or a protected relationship; 422 means invalid input. GETs can retry once on a network failure or 503. Writes are not automatically repeated, because a retry could duplicate a successful operation whose response was lost.

Updates are request-driven, not WebSocket push. Student tab switches reuse fetched data: reload the portal after an admin update. Only the SQL activity viewer polls automatically. An API failure after a write does not always prove nothing was saved; verify the current record before repeating it.

5:30–7:30 / APPLICATION WALKTHROUGH

5. Every administrator tab

Tab	Shows / lets admin do
Dashboard	Counts for students, faculty, departments, courses, active enrollments and semesters; department/admission/enrollment/semester charts; recent records and popular courses.
Students	Student identity, enrollment number, contact details, department, admission year and status. Manage records; open detail with enrollments.
Faculty	Employee identity, contact, designation and department; manage faculty and inspect their teaching assignments.
Departments	Academic unit codes/names/offices and related counts; manage organizational records.
Courses	Course code/title, credits and department. Maintain the catalog; details show its offerings.
Semesters	Academic year, dates and status; manage periods and inspect their scheduled offerings.
Course Offerings	A course in a specific semester/section, capacity, enrollment/free-seat figures and faculty; manage scheduled instances.
Faculty Assignments	Links teachers to offerings; manage teaching allocations.
Enrollments	Links students to offerings, with date/status and joined course information; create or update registrations.
Grades	Marks, maximum, letter and percentage linked to an enrollment; manage results.
Reports	Student sub-tab: department/year/status summaries. Faculty: department counts and assignments. Academic: enrollment, capacity, semester summaries and grade rows. CSV exports. No GPA feature on this screen.
Attendance	Enrollment ID, date and Present/Absent/Late/Excused; record and correct attendance.
Library books	ISBN, title, author and copies; maintain catalog records.
Library loans	Student/book IDs and borrowed/due/returned dates; issue a book and record its return. No automatic fine or stock enforcement.
Student access	Create a login linked to an existing student ID. Email is taken from that student; not a full account-edit/password-reset console.
SQL activity	Read-only recent query templates, request paths and timings; pause/resume polling or refresh. Not an arbitrary SQL editor.
Settings	Database health/path, local theme preference, counts, documentation and current SQL snapshot download.

Academic lists have configured search/sort/filter controls. Attendance and library endpoints currently implement SQL pagination and descending ID order, not the same full search/sort behavior. Deleting records with dependent history can be blocked.

6. Student portal and update demo

> “The student sees a deliberately smaller interface. Their job is to view their academic record, not edit official data. All four datasets are requested together when the portal loads, and the server filters them by the student ID linked to the authenticated account.”

Student tab	Information shown
Overview	Course-record count, total credits, attendance percentage, active-loan count; profile name/enrollment number/email/department; next outstanding book due date.
My courses	Course name/code, semester/year, section, credits, grade letter and marks obtained/maximum. A missing grade displays a dash.
Attendance	Date, course and recorded status; Present count and overall percentage.
Library	Own loan history: book title, author, ISBN, borrowed/due/returned dates; Returned, Overdue or On loan status.

Exact behavior worth knowing

- Attendance rate = rounded Present count ÷ all attendance records × 100. Late/Excused are not counted Present. No records shows a dash.
- Overview credits and course counts include all returned enrollment rows; current code does not remove Dropped/Completed rows from these calculations.
- Unreturned loans are active. The browser derives overdue status from the due date; it does not calculate a fine.
- Students cannot change their profile, grades, attendance, enroll themselves or issue books through this portal.

Quick demo sequence

1. Keep admin and student logged in using separate browser profiles/incognito; ordinary tabs share cookies.
2. As admin, edit an existing attendance record belonging to the demo student, e.g. Absent → Present.
3. Show the success toast and refreshed table.
4. Reload the student browser and show the changed row/percentage in Attendance.
5. Return to the admin SQL activity viewer and identify the matching PUT request.

Do not say that changing a student tab sends a new request: the portal currently loads /portal/me, /courses, /attendance and /library in parallel on mount or retry. A reload is the simplest way to show a new admin change.

7. SQL log: developer explanation

> “The SQL viewer makes backend integration visible. I can connect a button click to an HTTP request and the database statement it caused. It shows templates rather than private values. For example, an attendance edit produces an existing-row read, an UPDATE and a read-back of the updated row.”

```
PUT /api/attendance/selected-id
SELECT * FROM attendance WHERE attendance_id=?;
UPDATE attendance SET status=? WHERE attendance_id=?;
SELECT * FROM attendance WHERE attendance_id=?;

-- The form may submit additional fields in the UPDATE.
-- Snapshot reads and a separate list refresh can follow.
```

Action	What you can expect
Open academic list / search	Joined SELECT. Academic list filtering, sorting and paging happen in Node, so the SQL template may be unchanged when search text changes.
Create academic record	BEGIN IMMEDIATE → INSERT → joined detail read → COMMIT. Failure can cause ROLLBACK.
Edit / delete academic record	Read existing row, UPDATE or DELETE inside a transaction; edit reads the resulting detail. Constraints can reject a mutation.
Student portal load	SELECTs joining enrollment/course/grade, attendance/course or loan/book with WHERE student_id=? derived from the session.
Export SQL snapshot	Schema metadata, PRAGMA table_info and SELECT * reads. Generated INSERT text is written to a download/file, not executed against the database.
Change theme / switch student tab	Browser state only; no domain SQL needed.

Log columns: Time = capture time; Result = overall request outcome; Duration = statement time, not page latency; Request = method/path and grouping UUID; Role = requester; SQL template = masked statement; Rows = returned/changed rows, not rows scanned.

Limits: last 200 events in process memory, newest first; three-second polling; restart clears history. Pausing polling does not stop collection. Authentication/account/session queries are excluded. Trigger-internal statements are not individually traced. Failed request labels do not mean every SELECT in that request failed. Snapshot reads can crowd older events out of the buffer.

8:30–9:00 / SHORT DATABASE REFERENCE

8. Tables and keys — only the essentials

> “The database stores separate entities instead of repeating them in every row. A primary key uniquely identifies a record. A foreign key links records, such as a student to a department. Enrollment links a student to a specific offering; a grade belongs to that enrollment. This keeps the interface’s joined information consistent.”

Domain table	Primary key	Main foreign-key links
department	department_id	—
student	student_id	department_id → department
faculty	faculty_id	department_id → department
course	course_id	department_id → department
semester	semester_id	—
course_offering	offering_id	course_id → course; semester_id → semester
faculty_assignment	assignment_id	faculty_id → faculty; offering_id → offering
enrollment	enrollment_id	student_id → student; offering_id → offering
grade	grade_id	enrollment_id → enrollment
attendance	attendance_id	enrollment_id → enrollment
library_book	book_id	—
library_loan	loan_id	student_id → student; book_id → library_book

Four supporting tables: portal_account (PK account_id, linked student_id), portal_session (PK token_hash, linked account_id), login_throttle (PK key_hash) and portal_migration (PK name). Total: 12 domain + 4 support tables.

Constraints: unique identifiers stop duplicates; foreign keys protect relationships; check constraints validate allowed values; enrollment capacity triggers reject overbooking. Queries bind input through ? placeholders rather than concatenating user values.

Keep this section to 30 seconds. The table is for teacher questions, not something to read row by row. No ER diagrams are required for this developer-focused presentation.

9. Codebase management and closing

> “I separated screen components, shared UI, resource configuration, the HTTP client and server services so changes stay localized. A new field can be added through resource configuration without rebuilding a complete page. Shared loading and error components make failure behavior consistent. Git tracks feature changes; GitHub hosts the source. Local database files, secrets and personal-data snapshots stay out of version control.”

Successful domain changes persist in SQLite and also refresh a local full SQL snapshot. Settings can download a current domain-data snapshot; account/session data is excluded. The original schema/seed remains an initial setup source, not a file rewritten after every edit.

Verification includes Node API tests and Puppeteer browser tests. The recorded implementation run passed 14 Node tests and the production Next build; the portal smoke checked admin pages, student tabs and denied student access to admin endpoints. Test results are evidence from a run, not a promise that no bug can exist.

> “The main engineering result is a reusable frontend connected to a real backend, with clear role-specific experiences and predictable data flow. I can trace an action from component state to HTTP, validation, SQL and a refreshed UI. For production I would add deployment hardening, HTTPS, credential rotation, stronger operational monitoring and backup testing.”

Quick answers for likely developer questions

Question	Answer
React versus Next.js?	React renders components. Next provides routing, build tooling and server/API integration around React.
Why components instead of separate pages?	Shared components reduce repetition; configuration supplies entity differences. A shared fix improves multiple screens.
Frontend validation enough?	No. It improves feedback; the server must independently validate every request, and database constraints protect persisted integrity.
Where is authorization?	On the server in portal.mjs. Hiding admin navigation is only presentation, not the security boundary.
Why not retry POST automatically?	The write may already have succeeded even if its response was lost. Blind retries risk duplicate records.
Is this real-time?	Data is persisted immediately, but most UI updates require refetch/reload. SQL activity polls; no WebSocket push is implemented.
Why not an ORM?	Direct node:sqlite statements keep the local runtime simple and make the SQL explicit for demonstration.
Is SQLite only temporary memory?	No. This application uses a persistent on-disk database file. Only the audit buffer is in memory.

Repository: **github.com/Bhanu4417/DBMS-PROJCT**
Editable source: learnit/usecase.html · Presentation PDF: learnit/usecase.pdf